

**YENEPOYA INSTITUTE OF ARTS,
SCIENCE,
COMMERCE AND MANAGEMENT**

Project – Low Level Design

on

Behavioral Analytics for Energy Security

Student Name: Veekshith Poojary

Industry Mentor:
Mr.Sumith
Kumar Shukla

Guided
by:Divya N
Shetty

Table of Contents

- 1. Introduction**
- 2. Detailed Module Design**
- 3. Algorithm Design**
- 4. Data Structures**
- 5. Function-Level Design**
- 6. Error Handling**
- 7. Interface Design**
- 8. Performance Considerations**
- 9. References**

1. Introduction

1.1 Scope of Document

This document provides the detailed internal design of the Behavioral Analytics for Energy Security system. It covers the modular breakdown, algorithm logic, data structures, function-level specifications, error handling strategies, interface design, and performance considerations for implementation and review.

1.2 Intended Audience

- Developers
- Evaluators
- Project reviewers
- Security Operations Center (SOC) analysts

1.3 System Overview

The system is a web-based cybersecurity platform built with Python and Streamlit, specifically designed for the Energy Sector. It ingests user login log data, applies a Rule-Based Detection Engine for known threat signatures, and an unsupervised Machine Learning Engine (Isolation Forest) to detect unknown behavioral anomalies. It provides interactive dashboards, risk scoring, and exportable anomaly reports for security operators.

2. Detailed Module Design

2.1 Modules

1. Data Ingestion Module (src/data_loader.py)

- Accepts CSV file upload via Streamlit UploadedFile object or local file path
- Validates schema for required columns: user_id, login_time, status, location, device
- Returns a validated Pandas DataFrame or raises a descriptive error

2. Data Preprocessing Module (src/preprocessing.py)

- Drops rows with null user_id or login_time
- Parses login_time into Pandas datetime objects
- Strips whitespace and normalizes string fields
- Engineers temporal features: hour, day, weekday, month, is_failed

3. Rule-Based Detection Module (src/detector.py)

- Calculates per-user baselines: avg login hour, total logins, failed attempts, device history
- Applies 5 deterministic security rules to flag suspicious events
- Assigns quantitative risk scores (capped at 100) and categorizes into Low / Medium / High / Critical

4. Machine Learning Engine (src/ml_model.py)

- Trains an IsolationForest model (contamination=0.1) on engineered features
- Encodes categorical device field using LabelEncoder
- Serializes trained model and encoder to models/ directory as .pkl files
- Predicts outliers: -1 mapped to Suspicious, 1 mapped to Normal

5. Visualization Dashboard (src/dashboard.py)

- Generates 7 interactive Plotly charts with plotly_dark theme
- Includes: login hour histogram, daily trend, success/fail pie, risk distribution, top suspicious users, hour heatmap, PCA scatter

6. Reporting Utilities (src/utlis.py)

- Saves anomaly DataFrames to the reports/ directory as CSV
- Converts anomaly data to UTF-8 encoded string for Streamlit download button

7. Main Application (app.py)

- Entry point; manages Streamlit session state and UI orchestration
- Sidebar: file upload, threshold slider, ML toggle
- Triggers full analysis pipeline on button click
- Renders tabbed interface: Overview, Behavioral Analysis, Dashboard, Reports, About

3. Algorithm Design

3.1 Behavioral Anomaly Detection Algorithm

Steps:

1. Accept CSV login log from user or load default from sample_data/login_data.csv
2. Validate required schema columns (user_id, login_time, status, location, device)
3. Clean data: drop nulls, parse timestamps, normalize strings
4. Engineer temporal features: hour, day, weekday, month, is_failed
5. Compute per-user baseline: average login hour, total logins, failed count, known devices
6. Apply 5 rule-based security checks to every login event
7. Optionally train and run Isolation Forest on feature matrix
8. Merge rule-based and ML flags; compute and cap risk score at 100
9. Categorize risk: Low (0-30), Medium (31-60), High (61-80), Critical (81-100)
10. Render results in tabbed Streamlit dashboard; enable report export

3.2 Rule-Based Security Rules

Rule	Condition	Risk Points
Rule 1 – Suspicious Time	Login at exactly 3:00 AM	+40
Rule 2 – Time Deviation	Login more than 4 hrs from user's avg login hour	+25
Rule 3 – Failed Attempts	User total failed attempts exceed user-set threshold	+20 per failed event
Rule 4 – Frequency Spike	User has more than 10 total logins in dataset	+15
Rule 5 – Device Entropy	Login from a device not previously seen for that user	+15

3.3 Machine Learning Approach

- Algorithm: IsolationForest (Scikit-Learn), contamination=0.1
- Features used: hour, is_failed, total_logins, weekday, device_encoded
- device_encoded is derived via LabelEncoder on the categorical device column
- Output mapping: -1 (outlier) → Suspicious; +1 (inlier) → Normal
- PCA (2 components) used for 2D scatter visualization of 5D feature space

3.4 Risk Score Calculation Formula

Risk Score = Sum of all triggered rule points (capped at 100)

Score Range	Risk Level
0 – 30	Low
31 – 60	Medium
61 – 80	High
81 – 100	Critical

4. Data Structures

Data Entity	Structure	Description
Raw Login Dataset	Pandas DataFrame	CSV-loaded login records with columns: user_id, login_time, status, location, device
Engineered Feature Set	Pandas DataFrame	Cleaned data extended with hour, day, weekday, month, is_failed columns
User Patterns	Pandas DataFrame	Per-user aggregates: avg_hour, total_logins, failed_count, success_rate, device history

Data Entity	Structure	Description
Anomaly Results	Pandas DataFrame	Flagged events with is_anomaly bool, anomaly_reason string, risk_score int, risk_level string
Required Columns Schema	Python List	["user_id", "login_time", "status", "location", "device"]
ML Model	Joblib .pkl file	Serialized IsolationForest stored in models/isolation_forest.pkl
Label Encoder	Joblib .pkl file	Serialized LabelEncoder stored in models/device_encoder.pkl
Exported Report	CSV File	UTF-8 CSV of anomaly-only rows saved to reports/ directory

5. Function-Level Design

Module	Function	Input	Output	Description
data_loader.py	load_file(source)	File path or UploadedFile	DataFrame	Loads CSV; handles both local path and Streamlit upload objects
data_loader.py	validate_columns(df, required)	DataFrame, list	Boolean / Error	Ensures all required schema columns are present
preprocessing.py	clean_data(df)	Raw DataFrame	Cleaned DataFrame	Drops null rows, parses login_time, strips whitespace,

Module	Function	Input	Output	Description
				normalizes status
preprocessing.py	extract_features(df)	Cleaned DataFrame	Feature DataFrame	Derives hour, day, weekday, month, is_failed from login_time and status
detector.py	calculate_user_patterns(df)	Feature DataFrame	Patterns DataFrame	Groups by user_id to compute baselines and detect new_device events
detector.py	detect_rule_based(data, patterns, threshold)	DataFrames, int	Annotated DataFrame	Applies 5 security rules; sets is_anomaly and anomaly_reason per row
detector.py	calculate_risk(df)	Annotated DataFrame	Risk-scored DataFrame	Sums triggered rule weights; caps at 100; assigns risk_level category
ml_model.py	train_model(login_data)	Feature DataFrame	Trained IsolationForest, LabelEncoder	Encodes device, trains model, saves .pkl files to models/
ml_model.py	predict_anomalies(model, features)	Model, DataFrame	DataFrame with ml_flag column	Runs inference; maps -1 to

Module	Function	Input	Output	Description
				Suspicious and 1 to Normal
dashboard.py	plot_login_hour_distribution(df)	DataFrame	Plotly Figure	Histogram of login counts by hour (0-23)
dashboard.py	plot_daily_login_trend(df)	DataFrame	Plotly Figure	Line chart of logins per calendar day
dashboard.py	plot_success_failed_pie(df)	DataFrame	Plotly Figure	Pie chart of Successful vs Failed login ratio
dashboard.py	plot_risk_distribution(df)	DataFrame	Plotly Figure	Bar chart of event counts per risk level
dashboard.py	plot_top_suspicious_users(df)	DataFrame	Plotly Figure	Bar chart of top 10 users by anomaly event count
dashboard.py	plot_hour_heatmap(df)	DataFrame	Plotly Figure	2D heatmap: user_id vs hour vs login volume
dashboard.py	plot_anomalies_scatter(df)	DataFrame	Plotly Figure	PCA 2D scatter of Normal vs Suspicious ML-flagged events
utils.py	save_report(df, filepath)	DataFrame, path string	None (side effect)	Saves anomaly-only DataFrame as CSV to reports/ directory

Module	Function	Input	Output	Description
utils.py	export_csv(df)	DataFrame	UTF-8 string	Encodes DataFrame as UTF-8 CSV string for Streamlit download button

6. Error Handling

Error Condition	Module	Handling Strategy
Invalid or corrupted CSV file	data_loader.py	Try/except block around pd.read_csv(); displays Streamlit error message to user
Missing required schema columns	data_loader.py	validate_columns() raises ValueError with missing column names listed
Null/missing user_id or login_time	preprocessing.py	clean_data() silently drops rows with critical null values using dropna()
Unparseable login_time format	preprocessing.py	pd.to_datetime() with errors='coerce' converts failures to NaT; rows dropped downstream
No anomalies detected	app.py	UI displays informational message: 'No anomalies found'; empty report table shown gracefully
ML model .pkl file missing	ml_model.py	System trains a new model on the current dataset; no crash occurs
Division by zero in risk scoring	detector.py	Score uses additive logic only; no division operations; not applicable
Empty DataFrame after filtering	app.py	Conditional checks prevent rendering empty Plotly charts; fallback messages shown

Error Condition	Module	Handling Strategy
User uploads file with wrong delimiter	data_loader.py	pd.read_csv() default comma delimiter; error message shown if parsing fails

7. Interface Design

7.1 User Interface

- Web-based dashboard built with Streamlit (dark mode SOC-style theme via assets/style.css)
- Sidebar: CSV file upload widget, failed-attempts threshold slider (default 5, max 20), ML engine toggle checkbox
- Main panel: 'Run Behavioral Analysis' button triggers the full pipeline

7.2 Tabbed Output Interface

Tab	Contents
Overview	High-level KPI metrics: Total Users, Total Logins, Anomalies Found, High Risk Users; dataset preview table
Behavioral Analysis	Filterable data table of all flagged logins; bar chart of Risk Score distribution
Dashboard	7 interactive Plotly charts: hour histogram, daily trend, success/fail pie, risk distribution, top suspicious users, hour heatmap, PCA scatter
Reports	Table of anomaly-only events; Save to Server button (reports/); Download CSV button for browser download
About	Project documentation: purpose, detection logic, usage instructions

7.3 Visualization Screenshots

The following screenshots should be added here:

- Screenshot 1: Main sidebar with file upload, threshold slider, and ML toggle
- Screenshot 2: Overview tab showing KPI metric cards
- Screenshot 3: Behavioral Analysis tab with filterable anomaly table
- Screenshot 4: Dashboard tab showing all 7 Plotly charts
- Screenshot 5: Reports tab with download button

8. Performance Considerations

- All data processing is performed in-memory using Pandas and NumPy — no database required, enabling fast execution
- Isolation Forest training is performed once per session on button click; trained models are serialized to .pkl to avoid retraining on every run
- PCA dimensionality reduction applied only at visualization time, not during training, to reduce computational overhead
- Plotly charts are rendered client-side (JavaScript), minimizing server-side rendering cost
- Streamlit session state is used to persist processed DataFrames across tab switches without recomputation
- Default dataset (login_data.csv) provided in sample_data/ ensures the system is functional without user uploads
- Risk scoring uses pure additive arithmetic with no recursive or nested loops, ensuring $O(n)$ linear complexity
- Label encoding of device column is performed once during training and reused during inference via serialized encoder

9. References

- Streamlit Documentation — <https://docs.streamlit.io>
- Pandas Documentation — <https://pandas.pydata.org/docs>
- NumPy Documentation — <https://numpy.org/doc>
- Scikit-Learn Documentation (IsolationForest, PCA, LabelEncoder) — <https://scikit-learn.org/stable>
- Plotly Documentation — <https://plotly.com/python>
- Joblib Documentation — <https://joblib.readthedocs.io>
- Python Documentation — <https://docs.python.org>